

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 10%

QUESTIONS: 10 Total Marks:20 Annexure A

MOCK INTERVIEW QUESTIONS

Code of Conduct:

I Prudhvi Raj.B(192324274) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:prudhvi raj.b

Annexure A

1. You are working as a data analyst in a hospital where patient data is continuously collected and stored in CSV files. However, the dataset contains missing values, inconsistent units, and duplicate entries. The system must process this data in near real-time for reporting. Explain how you would design a solution using NumPy and Pandas to clean, filter, and standardize the data efficiently. Discuss indexing, aggregation, and saving the processed data, along with strategies to handle unexpected data anomalies.
2. A financial company provides you with millions of transaction records that must be analyzed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas DataFrames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.
3. You are given multiple datasets from different departments (sales, marketing, and customer support), each stored in different file formats. Your task is to combine these datasets into a unified DataFrame while ensuring no loss of critical information and maintaining consistent indexing. Explain your approach using Pandas for file I/O, combining, grouping, and sorting. How would you validate the merged dataset and resolve conflicts such as missing or duplicate keys?
4. A company requires an automated system that reads daily data files, processes them, and generates summary reports along with visualizations. The pipeline must filter relevant data, compute statistics, and produce sorted outputs for dashboards. Describe how you would design this workflow using NumPy and Pandas, including file handling, filtering, aggregation, and plotting. Also explain how you would handle errors such as corrupted files, inconsistent formats, or missing columns.
5. You are working on an e-commerce platform where customer behavior data is collected in real time. The dataset contains clickstream data with missing values and inconsistent formats. Explain how you would preprocess and structure this data using Pandas. Discuss filtering, feature extraction, indexing strategies, and how you would prepare the data for downstream machine learning tasks.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	25	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

MOCK INTERVIEW QUESTIONS – Annexure A

1. Hospital Patient Data – NumPy and Pandas

Interviewer: How would you design a solution to clean and standardize continuously collected hospital data?

Answer:

I would design a **near-real-time data preprocessing pipeline** using Pandas for DataFrame operations and NumPy for efficient numerical processing.

Steps:

1. **Read the CSV files**
 - Use `pd.read_csv()` to load incoming files.
 - For large files, process them in chunks using `chunksize`.
2. **Check the data**
 - Use `head()`, `info()`, `describe()` and `isnull().sum()` to identify problems.
3. **Handle missing values**
 - Numerical columns → replace missing values with median.
 - Categorical columns → replace with mode or "Unknown".
4. **Standardize units**
 - Convert measurements into a common unit.
 - For example, convert temperature values into Celsius if different units are present.
5. **Remove duplicates**
 - Use `drop_duplicates()` based on patient ID, timestamp, and record type.

Filter invalid values

Keep Age between 0 and 120

Keep valid blood pressure ranges

6. Remove impossible numerical values
7. **Use indexing**
 - Set patient ID or timestamp as an index when appropriate.
 - This makes lookup and time-based filtering faster.
8. **Aggregation**
 - Use `groupby()` to calculate statistics such as average blood pressure by department or patient category.
9. **Save processed data**
 - Save the cleaned DataFrame using `to_csv()` or another suitable format.
10. **Handle anomalies**
 - Detect unexpected values, wrong data types, duplicate records, and unusual measurements.
 - Log anomalies rather than silently deleting potentially important medical records.

Strong interview point:

For near-real-time processing, I would process only newly arrived files or records rather than repeatedly processing the entire historical dataset.

2. Financial Transactions – Large-Scale Analysis

Interviewer: How would you analyze millions of transactions under memory and computation constraints?

Answer:

I would use **NumPy** for efficient numerical calculations and **Pandas** for structured data processing, while minimizing unnecessary memory usage.

Steps:

1. Read only required columns using `usecols`.
2. Use appropriate data types such as `float32` or smaller integer types where precision allows.
3. Process very large CSV files in chunks.
4. Use Boolean indexing for efficient filtering.

Example:

```
transactions = transactions[transactions["amount"] > threshold]
```

Use `groupby()` for customer-level analysis:

GROUP transactions BY customer_id

5. CALCULATE total spending, average spending and transaction count
6. Use NumPy functions for statistical analysis:
 - Mean
 - Median
 - Standard deviation
 - Percentiles
7. Detect unusual spending using statistical thresholds or business rules.

Optimization

- Avoid unnecessary DataFrame copies.
- Filter data before grouping.
- Select only required columns.
- Use vectorized operations instead of Python loops.
- Use efficient indexes for frequently queried columns.
- Aggregate in chunks when the complete dataset cannot fit into memory.

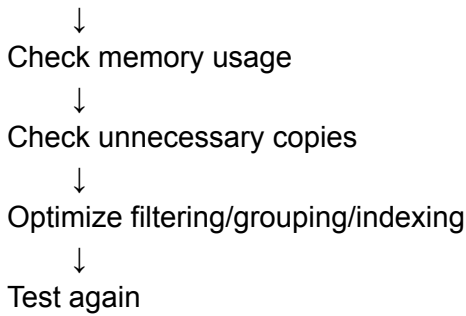
Debugging

If the program is slow, I would:

Measure execution time

↓

Identify slow operation



For incorrect results, I would check data types, missing values, duplicate transactions, filtering conditions, grouping keys, and sample records manually.

3. Combining Sales, Marketing and Customer Support Data

Interviewer: How would you combine datasets from different departments?

Answer:

First, I would inspect each dataset because they may have different formats, column names, data types, and keys.

Steps:

1. **Read different file formats**
 - CSV → `read_csv()`
 - Excel → `read_excel()`
 - JSON → `read_json()`

Inspect each DataFrame

Check columns

Check data types

Check missing values

2. Check duplicate keys

Standardize column names

"Customer ID" → "customer_id"

3. "Cust_ID" → "customer_id"
4. **Standardize key data types**
 - Ensure `customer_id` has the same type in every DataFrame.
5. **Handle missing keys**
 - Identify records without customer IDs.
 - Attempt correction when possible.
 - Preserve them separately if they cannot be safely matched.
6. **Remove or resolve duplicates**
 - Check duplicate customer IDs.

- Determine whether duplicates represent multiple valid events or actual duplicate records.

Merge datasets

Sales + Marketing + Customer Support



Common customer_id



7. Unified DataFrame

8. **Validate the merge**

- Compare record counts.
- Check unmatched records.
- Check duplicate keys.
- Check missing critical information.

9. **Group and sort**

- Group by customer or department.
- Calculate total sales, campaign responses, support requests, etc.
- Sort based on relevant metrics.

10. **Reset the index**

RESET index after final merge

Strong interview point:

I would use merge validation such as `validate="one_to_one"` or `validate="many_to_one"` where the expected relationship is known. This helps detect incorrect joins instead of allowing them silently.

4. Automated Daily Reporting Pipeline

Interviewer: How would you build an automated system that processes daily files and generates reports?

Answer:

I would design the system as an **ETL-style pipeline**:

Daily Files



File Validation



Data Loading



Cleaning



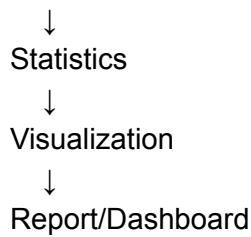
Filtering



Aggregation



Sorting



1. File Handling

- Check whether the daily file exists.
- Read it using Pandas.
- Use exception handling for corrupted files.

2. Data Validation

Check:

Required columns exist

Correct data types

Missing values

Duplicate records

Valid ranges

3. Data Cleaning

- Handle missing values.
- Convert data types.
- Standardize categorical values.
- Remove duplicate records.

4. Filtering

Apply business rules using Boolean conditions.

sales > 1000

AND

category = "Electronics"

5. Aggregation

Use `groupby()`:

GROUP BY category

CALCULATE total sales and average sales

6. Sorting

Sort the summary by the required metric:

`SORT total_sales` in descending order

7. Visualization

Use Matplotlib to create:

- Bar charts for category comparisons.
- Line charts for daily trends.
- Histograms for distributions.

8. Error Handling

If a file is corrupted:

TRY to read file

IF error:

- log error
- skip file
- continue pipeline

If required columns are missing:

IF required column does not exist:

- log the problem
- do not process that file

Strong interview point:

I would maintain logs for successful files, rejected files, row counts, and data-quality issues so that failures can be investigated without stopping the entire pipeline.

5. E-Commerce Clickstream Data

Interviewer: How would you preprocess real-time customer clickstream data for machine learning?

Answer:

I would use Pandas to convert raw clickstream records into a structured and machine-learning-ready DataFrame.

1. Load and Inspect Data

READ clickstream data

CHECK columns

CHECK data types

CHECK missing values

CHECK duplicate events

Typical fields could include:

user_id

timestamp

page

event_type

product_id

session_id
device

2. Handle Missing Values

- Fill categorical missing values with "Unknown".
- Fill numerical values using an appropriate statistic.
- Remove records when essential fields such as user ID or timestamp cannot be recovered.

3. Standardize Formats

Convert timestamp → datetime

Standardize event_type

Remove extra spaces

Convert categorical values to consistent format

4. Filtering

Remove invalid events:

timestamp must be valid

user_id must not be empty

event_type must be valid

5. Feature Extraction

Create useful ML features such as:

session_duration

number_of_clicks

pages_viewed

products_viewed

cart_additions

purchase_count

average_session_time

For example:

GROUP BY user_id and session_id

CALCULATE number of clicks

CALCULATE session duration

6. Indexing

I could use:

timestamp

user_id

session_id

as indexes depending on the queries required.

For time-based analysis, a datetime index can make filtering by time periods convenient.

7. Prepare for Machine Learning

Clean Data



Feature Extraction



Encode Categorical Variables



Scale Numerical Features if required



Handle Missing Values



Remove Data Leakage



Train/Test Split



ML Model

8. Final Validation

Before sending the data to the ML model, I would verify:

- No unexpected missing values.
- Correct data types.
- No duplicate events.
- Valid timestamps.
- Correct feature ranges.
- No target leakage.

Strong interview point:

Because clickstream data is time-dependent, I would be careful with train/test splitting. A time-based split is often more appropriate than a random split when the goal is to predict future customer behavior.